



MVPFY

MVPFy — Guia do usuário · v0.4.0

Defina o primeiro MVP SaaS

Percurso do usuário para transformar uma ideia em um plano de MVP SaaS.

Guia do usuário

Edição 0.4.0

Gerado em 16/08/2026

Fonte editorial: docs/user/

Sumário

Guia completo do usuário	6
Leia online ou como ebook	6
O que o MVPFy resolve	6
Percurso pedagógico	7
Conversa contínua	7
Comece	7
A proposta do MVPFy	8
O resultado que você recebe	8
O que não faz parte	8
Por que SaaS é um recorte necessário	8
O princípio da versão 1.0	9
Uma distinção importante	9
Instalação em um projeto consumidor	10
Instalar as skills	10
Arquivos criados	10
Conferir a instalação	10
Atualização	11
Primeiro MVP: um percurso completo	12
1. Comece com uma frase	12
2. Responda uma pergunta por vez	12
3. Dê respostas ricas quando quiser	13
4. Defina a jornada principal	13
5. Defina o SaaS	14
6. Pesquise mercado e preço	14
7. Gere o MVP.md	14
8. O que você entrega às equipes	14
Entrevista adaptativa	15
Entrada inicial	15
O ciclo de cada resposta	16
Formato fixo de cada turno	16
Formas de resposta	16
Comandos de conversa	17
Como o fluxo evita perguntas repetidas	17

Quando o MVP está grande demais.....	17
O arquivo <code>MVP.md</code>	18
Estados do documento.....	18
Estrutura canônica	18
Fato, recomendação e hipótese.....	19
Atualização sem perda	19
Pesquisa de mercado, preço e custo.....	20
Com URLs de concorrentes.....	20
Sem concorrentes claros.....	20
Como o preço é montado.....	20
Tecnologia e operação recomendadas.....	22
Componentes de referência	22
O que precisa ser decidido no MVP.....	22
Operação assistida.....	23
Solução de problemas	24
A instalação não cria o estado	24
A pergunta seguinte repetiu uma resposta	24
O documento está como <code>preliminary</code>	24
Uma escolha anterior mudou	24
O ebook está desatualizado	24
Catálogo de skills do MVPFy	25
Ordem típica.....	25
Skills.....	25
Skill <code>mvpfy</code>	27
Quando usar	27
O que ela lê	27
O que ela faz.....	27
Regra rígida de saída.....	27
Limite sobre fontes externas ao MVPFy	28
Skill <code>mvpfy-problem</code>	29
Analisa.....	29
Saída	29
Exemplo	29
Não faz.....	29

Skill <code>mvpfy-audience</code>	30
Analisa	30
Saída	30
Exemplo	30
Não faz	30
Skill <code>mvpfy-product</code>	31
Analisa	31
Classificação	31
Exemplo	31
Não faz	31
Skill <code>mvpfy-saas</code>	32
Analisa	32
Exemplo	32
Recomendação padrão	32
Não faz	32
Skill <code>mvpfy-brand</code>	33
Entrega	33
Exemplo	33
Não faz	33
Skill <code>mvpfy-market</code>	34
Analisa	34
Saída	34
Exemplo	34
Não faz	34
Skill <code>mvpfy-pricing</code>	35
Analisa	35
Saída	35
Exemplo	35
Não faz	35
Skill <code>mvpfy-technology</code>	36
Base recomendada	36
Entrega	36
Exemplo	36
Não faz	36

Skill <code>mvpfy-marketing</code>	37
Entrega.....	37
Exemplo	37
Não faz.....	37
Skill <code>mvpfy-document</code>	38
Arquivos usados.....	38
Operação.....	38
Não faz.....	38
Skill <code>mvpfy-migrate</code>	39
Fluxo	39
Exemplo	39
Não faz.....	39

Guia completo do usuário



O MVPFy ajuda você a sair de uma ideia vaga e chegar a um plano claro para o primeiro software oferecido como serviço. Você conversa normalmente com a skill `mvpfy`; ela faz uma pergunta por vez, guarda cada resposta e aciona os especialistas adequados para produto, mercado, preço, tecnologia e marketing.

O resultado é um `MVP.md` que outras pessoas conseguem usar para criar o website, desenhar a marca, construir a versão 1.0, vender o serviço e operar os primeiros clientes.

O MVPFy não cria `spec.md`, não executa o método do Specsify e não altera o repositório `specsify`. A semelhança entre os guias existe apenas na organização da documentação e no padrão visual do ebook.

Leia online ou como ebook

Este percurso integra a edição portátil **v0.4.0**:

- [PDF](#), para leitura e impressão;
- [EPUB](#), para leitores digitais;
- [manifesto](#), com a versão e os hashes do build.

O ebook contém somente este percurso do usuário. A ordem de compilação está em [reading-order.txt](#). A referência técnica e a especificação continuam disponíveis na documentação online para agentes e contribuidores.

O que o MVPFy resolve

No começo de uma empresa, a ideia costuma misturar público, problema, funcionalidades, preço e tecnologia. O MVPFy organiza essa conversa sem pedir que você já conheça termos como ICP, unit economics ou multitenancy.

O recorte é específico:

- uma empresa nova ou uma operação ainda em formação;
- o primeiro produto é um software;
- o software será oferecido como serviço;
- o trabalho atual é definir a versão 1.0, não todo o produto futuro.

Se a proposta for uma agência que cria sites sob encomenda, o MVPFy pergunta se existe também um produto SaaS repetível. Sem esse produto, o projeto está fora do recorte principal.

Percurso pedagógico

1. Leia [A proposta](#) para entender o que entra e o que fica fora.
2. Siga [Instalação](#) para preparar um projeto consumidor.
3. Execute [Primeiro MVP](#) com um exemplo completo.
4. Consulte [Entrevista](#) para pausar, continuar e corrigir.
5. Veja [MVP.md](#) para entender o documento produzido.
6. Use [Pesquisa e preço](#) e [Tecnologia e operação](#) quando essas áreas entrarem na conversa.
7. Consulte o [catálogo de skills](#) para entender cada especialista.

Conversa contínua

Você não precisa responder tudo em uma sessão. O MVPFy salva o estado no projeto consumidor. Quando você disser "continuar", ele lê o que já existe, identifica a pendência mais importante e faz somente a próxima pergunta.

Uma resposta rica também pode preencher várias áreas. Se você disser que agências pequenas perdem leads recebidos por WhatsApp, formulário e Instagram, o MVPFy pode aproveitar a frase para problema, público, canais e alternativa atual sem perguntar tudo novamente.

Comece

Para aprender fazendo, abra [Primeiro MVP](#). Para consultar uma regra específica, use o índice de [skills](#) ou o [guia de desenvolvimento](#).

A proposta do MVPFy

O MVPFy existe para uma situação específica: você está montando uma empresa do zero e precisa transformar a primeira ideia de software em um MVP SaaS que possa ser construído, vendido e aprendido com clientes reais.

O resultado que você recebe

Ao terminar a entrevista, o MVPFy gera um único arquivo chamado `MVP.md`. Ele funciona como uma visão compartilhada para produto, desenvolvimento, website, marca, marketing, vendas e operação. O arquivo não é uma transcrição da conversa. Ele separa o que você informou, o que foi recomendado, o que ainda é hipótese e o que precisa ser confirmado.

O documento cobre:

- empresa, contexto e modelo SaaS;
- problema, alternativa atual e hipótese de valor;
- público, comprador, pagador, usuários e personas;
- proposta, jornada principal e escopo da versão 1.0;
- conta, onboarding, ativação, assinatura, suporte e cancelamento;
- marca, nome, slogan e posicionamento inicial;
- concorrentes, preços observados e alternativas manuais;
- preço, custo operacional, receita recorrente e margem;
- Laravel, VPS, banco, arquivos, e-mail, IA e operação;
- website, conteúdo, captação, vendas e lançamento;
- métricas, validações, pendências e plano de execução.

O que não faz parte

O MVPFy não constrói o software, não publica o website, não cria um logo final e não promete resultado comercial. Ele prepara o plano para que outros agentes ou equipes executem essas tarefas.

Também não cria `spec.md`, não altera o Specsfy e não converte o `MVP.md` em uma especificação do Specsfy. O produto tem seu próprio artefato, seu próprio estado e seus próprios scripts.

Por que SaaS é um recorte necessário

Um software oferecido como serviço exige mais do que uma lista de funcionalidades. O plano precisa explicar quem cria a conta, quem usa, quem paga, como a pessoa chega ao primeiro valor, como o acesso é mantido e como o serviço é encerrado.

Durante a entrevista, o MVPFy pode recomendar processos manuais para validação. Por exemplo, uma pessoa pode liberar uma conta, acompanhar a implantação ou conferir um pagamento sem que a primeira versão automatize tudo. Essa escolha fica registrada para que a equipe saiba o que é operação provisória e o que é requisito do produto.

O princípio da versão 1.0

O MVPFy procura manter uma combinação simples: um público prioritário, um problema principal, uma promessa central e uma jornada que entrega essa promessa de ponta a ponta. Recursos adicionais entram em “Fora do MVP e evolução futura” quando não são necessários para validar o produto.

Uma distinção importante

O MVPFy pode sugerir Laravel, VPS, Supabase, armazenamento de objetos e um provedor de IA. Essas são recomendações iniciais baseadas no padrão da Promovaweb, não decisões irrevogáveis. O projeto final precisa registrar a razão de cada escolha e o gatilho para revisá-la.

Instalação em um projeto consumidor

O MVPFy é instalado no projeto que vai receber o planejamento. O repositório `mvpfy` guarda as skills e os scripts; o estado da entrevista fica no projeto consumidor.

Instalar as skills

No projeto consumidor, execute:

```
npx skills add promovaweb/mvpfy
```

Depois, inicie a estrutura local:

```
node skills/mvpfy/scripts/setup-project.mjs --project .
```

O comando prepara `.mvpfy/` e cria `MVP.md` a partir do template. Ele não altera código da aplicação, não cria `spec.md` e não escreve no repositório Specsify.

Arquivos criados

```
projeto/  
├─ MVP.md  
└─ .mvpfy/  
    ├─ config.yaml  
    ├─ state.json  
    ├─ answers.jsonl  
    └─ research.json
```

`MVP.md` é o arquivo que você compartilha com as equipes. `.mvpfy/` guarda o estado operacional para retomar a entrevista, registrar respostas e manter pesquisas. O diretório `.mvpfy/` mantém o estado para a retomada da entrevista, para os campos e as regras de gravação.

Conferir a instalação

No repositório do MVPFy, a suíte pode ser executada com:

```
npm test
```

Em um projeto consumidor, carregue `$mvpfy` e peça para começar. A primeira resposta deve ser salva antes da pergunta seguinte. Se a estrutura não for criada, confira [Solução de problemas](#).

Atualização

Atualize as skills pelo mesmo mecanismo usado na instalação. Ao encontrar um `MVP.md` de uma versão anterior, a orquestradora compara o template e usa `mvpfy-migrate` antes de fazer novas perguntas. A atualização preserva as respostas existentes.

Primeiro MVP: um percurso completo

Este exemplo é didático. Ele não descreve um produto real do MVPFy. A ideia é uma ferramenta para pequenas agências acompanharem leads recebidos por WhatsApp, formulário e Instagram.

1. Comece com uma frase

Você pode iniciar assim:

```
Quero criar um sistema para pequenas agências que perdem leads dos clientes.
```

Ou pode enviar um contexto completo de uma vez:

```
Quero um SaaS para pequenas agências acompanharem leads dos clientes. A agência paga, o cliente acompanha apenas os próprios contatos e o MVP deve receber o lead, atribuí-lo a alguém e mostrar um resumo. Já imagino CRM, integração com WhatsApp e um painel. Quero usar Laravel em uma VPS e cobrar mensalidade.
```

O agente analisa esse texto inteiro antes de perguntar. CRM, WhatsApp e painel ficam como candidatos para investigação, enquanto problema, público, jornada, tecnologia e cobrança já podem preencher partes do plano. Você pode mandar mais mensagens antes de escolher `4. Continuar para as perguntas`.

O MVPFy não transforma essa frase diretamente em módulos. Ele começa entendendo a situação.

2. Responda uma pergunta por vez

Uma conversa possível:

MVPFy: Em qual situação o problema aparece com mais frequência?

1. A equipe recebe contatos em vários canais e perde o acompanhamento.
2. A equipe demora para responder ao primeiro contato.
3. A equipe não consegue mostrar resultados para o cliente.
4. Avançar
5. Conversar mais sobre este tema

Você: 1

MVPFy: resposta salva. Quem usaria o produto no trabalho diário?

1. A equipe da agência.
2. O cliente da agência.
3. A equipe e o cliente, com acessos diferentes.
4. Avançar
5. Conversar mais sobre este tema

O texto “resposta salva” representa o comportamento obrigatório. A pergunta seguinte só aparece depois do registro no `answers.jsonl` e da atualização do estado.

Se você escolher 5, pode explicar um caso, perguntar o significado de uma opção ou trazer uma resposta mais complexa. O MVPFy conversa sobre a mesma questão, salva cada turno e continua exibindo no máximo uma pergunta por vez.

3. Dê respostas ricas quando quiser

Você não precisa escolher apenas um número. Esta resposta também é válida:

A agência paga. Cada cliente deve acompanhar somente os próprios leads, e a equipe da agência precisa administrar tudo em uma conta.

Essa frase pode preencher comprador, pagador, titular da conta, usuários, permissões e isolamento entre clientes. O MVPFy não deve perguntar novamente quem paga.

4. Defina a jornada principal

Depois de entender o problema, a conversa reduz o produto a um fluxo. No exemplo, uma jornada possível é:

1. A agência cria uma conta.
2. A equipe cadastra um cliente.
3. O sistema recebe ou registra um lead.
4. A equipe atribui o lead a alguém.

5. O cliente acompanha o andamento.

6. A agência mostra o resultado em um resumo simples.

O MVPFy pergunta quais etapas são indispensáveis. Um aplicativo móvel, um marketplace e automações avançadas podem ficar para depois se não forem necessários para provar essa jornada.

5. Defina o SaaS

O plano precisa registrar que a agência é a conta pagadora, que cada cliente tem acesso limitado e que o primeiro valor ocorre quando um lead é registrado, atribuído e acompanhado. Também precisa dizer se o teste é gratuito, se a implantação será assistida e qual evento confirma ativação.

6. Pesquisa mercado e preço

Se você fornecer URLs de concorrentes, o especialista de mercado consulta planos, limites e preços atuais e registra fonte e data. Se não houver uma referência direta, a conversa pergunta quanto a agência gasta hoje com planilha, horas de acompanhamento e perda de leads.

O especialista de preço transforma esses dados em uma faixa inicial, não em uma promessa. O documento pode trazer um cenário enxuto, um cenário base e um cenário de crescimento.

7. Gere o MVP.md

Quando você pedir “gerar o documento”, o MVPFy consolida todas as áreas. Uma versão inicial pode ter estado `preliminary` e mostrar `Pendente` nas lacunas. Quando houver problema, público, jornada, escopo, modelo SaaS, preço, tecnologia, aquisição, métricas e validações suficientes, o documento pode passar a `ready`.

8. O que você entrega às equipes

O `MVP.md` permite que cada equipe comece do mesmo entendimento:

- produto recebe escopo e critérios de aceite;
- desenvolvimento recebe arquitetura, contas e integrações;
- website recebe público, promessa e oferta;
- marca recebe posicionamento, nomes e tom;
- marketing recebe canais, conteúdo e captação;
- vendas recebe preço, processo e objeções;
- operação recebe onboarding, suporte e métricas.

Entrevista adaptativa

A entrevista é a interface principal do MVPFy. Você fala em linguagem comum e a orquestradora escolhe a próxima lacuna com maior impacto no plano.

Entrada inicial

Antes da primeira pergunta com opções, descreva livremente a ideia do SaaS e do MVP. Conte o que o produto pode fazer, para quem ele serviria e quais módulos, recursos ou integrações você já imaginou. Essa é a única entrada livre de abertura. O MVPFy salva o texto e separa os itens citados como candidatos.

Depois disso, começa a entrevista com uma pergunta por vez e cinco opções. Um item citado não entra automaticamente na versão 1.0: ele será investigado e poderá entrar no MVP, ficar para depois ou ser descartado.

Você também pode enviar tudo em uma mensagem. Por exemplo:

```
Quero um SaaS para pequenas agências acompanharem leads de seus clientes.  
Hoje elas usam planilhas e WhatsApp. A agência paga, cada cliente vê apenas  
seus próprios leads e o MVP precisa cadastrar o lead, atribuí-lo a alguém e  
mostrar um resumo. Quero começar com Laravel em uma VPS e cobrar mensalidade.
```

Nesse caso, o MVPFy registra problema, público, comprador, permissões, jornada, tecnologia e cobrança antes de perguntar. Ele não pergunta novamente quem paga nem quais etapas já foram descritas. A próxima pergunta trata apenas da lacuna mais importante que restar.

Se ainda quiser acrescentar informações, envie outra mensagem. Todas são acolhidas e salvas. Quando terminar, escolha 4. Continuar para as perguntas.

O encerramento da entrada inicial pode aparecer assim:

```
MVPFy: Entendi a ideia e já registrei o problema, o público, a jornada e os  
itens candidatos. Você quer acrescentar algo antes de começar?  
1. Acrescentar outro módulo ou recurso.  
2. Explicar melhor o público ou o problema.  
3. Informar tecnologia, preço ou canal de venda.  
4. Continuar para as perguntas.  
5. Conversar mais sobre este tema.
```

As opções 1, 2, 3 e 5 permitem continuar enviando conteúdo. A opção 4 salva o estado da entrada e libera a próxima pergunta fechada.

O ciclo de cada resposta

1. Você responde com número, texto, combinação de opções ou “não sei”.
2. O MVPFy guarda a resposta original.
3. A resposta é normalizada sem alterar seu sentido.
4. Fatos, escolhas, hipóteses e áreas cobertas são extraídos.
5. `state.json` e `MVP.md` são atualizados quando aplicável.
6. A orquestradora recalcula o que ainda falta.
7. Uma única pergunta seguinte é apresentada.

Se a gravação falhar, o fluxo não avança. A mesma pergunta volta para que você possa tentar novamente.

Formato fixo de cada turno

Cada pergunta aparece com cinco opções:

```
Qual situação descreve melhor o problema?  
1. A equipe perde o acompanhamento.  
2. A equipe demora para responder.  
3. A equipe não consegue mostrar o resultado.  
4. Avançar  
5. Conversar mais sobre este tema
```

As três primeiras opções são respostas prontas para a mesma pergunta. A opção 4 aceita o entendimento atual e leva à próxima etapa. A opção 5 abre um chat sobre o mesmo tema: você pode escrever uma explicação, uma dúvida ou um caso complexo. O MVPFy pode investigar esse caso em mais de um turno, mas continua exibindo no máximo uma pergunta por vez e salva cada resposta antes de seguir.

Nunca aparecem duas perguntas principais no mesmo turno.

Formas de resposta

Você pode responder:

- 1, 2 ou 3;
- 4 para avançar;
- 5 para conversar mais sobre a pergunta atual;
- “2, mas também atendo consultores”;
- uma explicação livre;

- “ainda não sei”;
- uma correção de resposta anterior;
- “pausar”, “continuar”, “revisar preço” ou “gerar documento”.

“Ainda não sei” também é informação. O MVPFy registra a lacuna e pode sugerir uma hipótese provisória, deixando claro que ela precisa de validação.

Comandos de conversa

PEDIDO	RESULTADO
“Começar”	Cria ou abre um projeto e faz a primeira pergunta útil.
“Continuar”	Lê o estado e retoma pela pendência prioritária.
“Pausar”	Salva o ponto atual sem apagar respostas.
“Revisar preço”	Trabalha somente em modelo, faixa e premissas de preço.
“Mudar o público”	Registra a correção e revisa áreas dependentes.
“O que falta?”	Mostra um resumo das pendências sem abrir formulário.
“Gerar documento”	Consolida a melhor versão, mesmo que preliminar.

Como o fluxo evita perguntas repetidas

Cada resposta recebe uma lista de campos cobertos. Uma resposta sobre “a agência paga e o cliente acompanha” cobre papéis comerciais e de produto. A orquestradora usa esses campos para retirar perguntas redundantes da fila.

Ela também verifica contradições. Se antes o comprador era uma clínica e depois você disser que venderá para agências, a alteração é registrada como correção. As personas, preço, jornada e canais relacionados voltam para revisão.

Quando o MVP está grande demais

O MVPFy pede uma escolha quando encontra várias jornadas, muitos públicos ou módulos independentes. A pergunta não é “quais recursos você quer?”, mas:

Qual resultado precisa funcionar do começo ao fim na primeira versão?

1. Registrar e acompanhar o lead.
2. Gerar o relatório para o cliente.
3. Automatizar a distribuição do lead.

As outras ideias são preservadas em “Fora do MVP e evolução futura”.

O arquivo MVP.md

MVP.md é a entrega central do MVPFy. Ele deve ser compreensível sem o histórico da conversa e útil para equipes diferentes trabalharem no mesmo MVP.

Estados do documento

ESTADO	SIGNIFICADO
preliminary	Há conteúdo aproveitável, mas faltam escolhas importantes.
ready	Os campos mínimos do primeiro SaaS estão descritos e coerentes.
validated	As hipóteses principais já foram conferidas fora da entrevista.

O estado não é uma nota de qualidade. Ele mostra quanto do plano já foi confirmado.

Estrutura canônica

O template atual contém 35 áreas:

1. resumo executivo;
2. empresa e contexto;
3. modelo da empresa SaaS;
4. problema;
5. comprovação, alternativas e hipóteses;
6. público-alvo;
7. usuário, comprador e pagador;
8. personas;
9. proposta de valor e posicionamento;
10. nome, marca e slogan;
11. jornada principal;
12. conta, acesso e isolamento;
13. onboarding e primeiro valor;
14. escopo funcional 1.0;
15. módulos e funcionalidades;
16. perfis e permissões;
17. assinatura e cobrança;
18. suporte, retenção e cancelamento;
19. processos manuais;

20. fora do MVP;
21. concorrentes e alternativas;
22. modelo comercial e preço;
23. custos e economia unitária;
24. tecnologia e arquitetura;
25. infraestrutura e operação;
26. uso de IA;
27. website;
28. marketing;
29. vendas e lançamento;
30. métricas;
31. validações e dependências;
32. execução;
33. escolhas confirmadas;
34. hipóteses e pendências;
35. fontes.

Cada seção possui um comentário estável como `<!-- mvpfy:section:problem-definition -->`. O título pode mudar sem fazer o renderer perder o conteúdo.

Fato, recomendação e hipótese

O documento usa rótulos claros:

- **Confirmado:** informação declarada por você ou comprovada por uma fonte.
- **Recomendado:** sugestão feita por uma especialista com base no contexto.
- **Hipótese:** afirmação ainda não conferida com pessoas, mercado ou uso.
- **Pendente:** dado necessário que ainda não foi fornecido.

Essa separação impede que uma estimativa de preço pareça uma pesquisa concluída ou que uma sugestão de Laravel pareça uma exigência do produto.

Atualização sem perda

Quando o template evolui, `mvpfy-migrate` insere seções novas, preserva o texto existente e marca os campos que precisam de resposta. Depois da migração, a orquestradora faz apenas uma pergunta nova por vez.

Pesquisa de mercado, preço e custo

O MVPFy pesquisa o mercado quando você informa concorrentes, URLs ou pede uma referência atual. A pesquisa apoia a conversa; ela não substitui a validação com compradores.

Com URLs de concorrentes

Informe uma URL ou nome. A skill `mvpfy-market` procura, quando disponível:

- público e promessa;
- recursos relevantes para comparação;
- preço mensal, anual ou sob consulta;
- moeda, limites e condições do plano;
- diferença entre concorrente direto, indireto e alternativa manual.

O `MVP.md` registra fonte e data. Preço ausente continua como “não publicado”; o MVPFy não inventa uma faixa e não trata promoção como preço permanente.

Sem concorrentes claros

A conversa investiga a alternativa usada hoje. Esses pontos podem entrar na fila, mas aparecem um por turno, sempre no formato de três opções, `Avançar` e `Conversar mais sobre este tema`:

- tempo gasto por mês;
- ferramentas combinadas;
- erros, perdas ou atrasos;
- resultado que teria valor suficiente para pagar.

Essa informação serve como referência de valor percebido e ajuda a escolher uma unidade de cobrança simples.

Como o preço é montado

`mvpfy-pricing` considera comprador, unidade de valor, frequência de uso, benefício, esforço comercial, custo variável, suporte e estágio da validação. Pode recomendar cobrança por conta, usuário, volume, uso ou um modelo híbrido.

O plano deve mostrar premissas e três cenários quando houver dados:

CENÁRIO	USO
Enxuto	Poucos clientes, infraestrutura mínima e operação assistida.
Base	Volume inicial esperado e custos recorrentes conhecidos.

CENÁRIO	USO
Crescimento	Aumento de uso sem introduzir complexidade antes da hora.

As contas conceituais são:

```

custo mensal = fixos + variáveis + IA + comunicação + suporte + taxas
margem por cliente = receita líquida - custo variável por cliente
clientes para equilíbrio = custos fixos / margem por cliente
MRR = soma da receita recorrente mensal dos clientes ativos
ARPA = MRR / contas pagantes

```

O resultado é uma faixa com premissas. Não é uma previsão contábil nem uma garantia de receita.

Tecnologia e operação recomendadas

O padrão inicial da Promovaweb para os projetos atendidos pelo MVPFy é Laravel em uma VPS. A recomendação favorece uma aplicação simples, custo previsível e um caminho curto até a primeira validação.

Componentes de referência

NECESSIDADE	PADRÃO INICIAL
Aplicação	Laravel.
Serviço complementar	Node.js somente com justificativa concreta.
Banco	PostgreSQL, com Supabase quando fizer sentido ao projeto.
Servidor	VPS com ambientes e backups definidos.
Arquivos	Object storage compatível com S3.
Mensagens	Provedor transacional de e-mail.
IA	Provedor externo apenas quando houver função de produto clara.
Operação	Logs, backup, filas ou tarefas agendadas conforme a jornada.

O especialista pode recomendar outra opção quando o contexto exigir, mas deve explicar o motivo e o efeito no custo e na operação.

O que precisa ser decidido no MVP

O plano não precisa desenhar cada classe do sistema. Ele precisa esclarecer:

- como a conta é criada e identificada;
- como os dados de clientes ficam separados;
- quem pode acessar cada parte;
- quais integrações são indispensáveis;
- quais tarefas podem ser assíncronas;
- onde arquivos são guardados;
- como backup, logs e suporte funcionam;
- qual custo mensal aparece em cada cenário.

Para a maioria dos primeiros SaaS, o MVPFy prefere uma aplicação compartilhada com separação lógica segura entre contas. Instâncias dedicadas só entram com uma justificativa de contrato, segurança, desempenho ou posicionamento.

Operação assistida

Durante a validação, liberar conta, acompanhar onboarding ou conferir uma assinatura manualmente pode ser aceitável. O `MVP.md` registra o processo, a pessoa responsável e o momento em que a automação passa a valer a pena.

Solução de problemas

A instalação não cria o estado

Confirme que o comando foi executado na raiz do projeto consumidor:

```
node skills/mvpfy/scripts/setup-project.mjs --project .
```

Depois confira se existem `MVP.md` e `.mvpfy/state.json`. O comando deve ser executado dentro do consumidor, não na raiz do repositório `mvpfy` para alterar o próprio pacote.

A pergunta seguinte repetiu uma resposta

Confira se a resposta anterior foi salva no `answers.jsonl` e se os campos extraídos foram atualizados no `state.json`. Uma resposta com texto livre pode precisar de interpretação; ela não deve ser descartada só porque não contém um número.

O documento está como preliminary

Abra a seção “Hipóteses e pendências”. Esse estado significa que falta pelo menos um item necessário para descrever o primeiro SaaS. Peça “continuar” ou “o que falta?” para voltar à pendência prioritária.

Uma escolha anterior mudou

Diga claramente a alteração, por exemplo: “o público agora são clínicas, não agências”. O MVPFy preserva a resposta anterior como histórico, registra a nova resposta e revisa as áreas afetadas.

O ebook está desatualizado

No repositório `mvpfy`, execute:

```
npm run ebook
npm run ebook:verify
```

Se uma página estiver ausente, confira o caminho correspondente em `docs/user/reading-order.txt`.

Catálogo de skills do MVPFy

O usuário conversa com `$mvpfy`. A orquestradora escolhe a especialista e integra o retorno. Você não precisa chamar cada skill manualmente, mas conhecer as responsabilidades ajuda a entender por que uma pergunta apareceu.

Ordem típica

```
contexto existente
  ↓
problema → público → produto → SaaS
  ↓         ↓         ↓         ↓
mercado → preço → tecnologia → marketing
  ↓
marca → MVP.md → migração quando o template mudar
```

Essa ordem é adaptativa. Uma fonte já existente pode preencher uma área inteira ou fazer a orquestradora voltar a um domínio anterior.

Skills

SKILL	QUANDO ENTRA	ENTREGA PRINCIPAL
mvpfy	Sempre que você inicia ou retoma	Próxima pergunta e estado integrado
mvpfy-problem	O problema ainda está genérico	Declaração do problema e hipótese
mvpfy-audience	Público ou papéis estão misturados	Segmento, ICP e personas
mvpfy-product	A solução precisa virar escopo	Jornada, módulos e versão 1.0
mvpfy-saas	É preciso explicar o serviço recorrente	Conta, onboarding e ciclo do cliente
mvpfy-brand	Produto e público já têm direção	Nome, posicionamento e slogan
mvpfy-market	Há concorrentes ou URLs	Mapa competitivo e fontes
mvpfy-pricing	Valor e cobrança precisam de hipótese	Faixas, planos e cenários
mvpfy-technology	A jornada permite desenhar a base técnica	Arquitetura Laravel e custos
mvpfy-marketing	Oferta e público precisam chegar ao mercado	Aquisição, conteúdo e venda
mvpfy-document	Você pede o plano ou há dados suficientes	<code>MVP.md</code> renderizado e validado

SKILL	QUANDO ENTRA	ENTREGA PRINCIPAL
mvpfy-migrate	O template mudou	Documento atualizado sem perda

Cada página explica entradas, análise, saída, limites, exemplo e próximo handoff. O catálogo descreve o MVPFy; ele não altera a biblioteca Specsify.

Skill `mvpfy`

`mvpfy` é a orquestradora e o único ponto de conversa. Ela não substitui o conhecimento das especialistas. Ela lê o contexto existente, escolhe o domínio necessário, controla a entrevista e consolida o resultado.

Quando usar

Use `$mvpfy` para começar, continuar, pausar, corrigir uma escolha, revisar uma área ou gerar o `MVP.md`.

O que ela lê

- `.mvpfy/config.yaml`;
- `.mvpfy/state.json`;
- `.mvpfy/answers.jsonl`;
- `MVP.md`;
- template atual;
- specs, backlogs, briefs, docs, tickets e planos já existentes no projeto consumidor, sempre em modo somente leitura.

O que ela faz

1. Confere se o projeto é um primeiro SaaS.
2. Reaproveita informação já disponível.
3. Verifica se o template precisa de migração.
4. Escolhe uma lacuna prioritária.
5. Exibe exatamente uma pergunta principal.
6. Registra a resposta antes de prosseguir.
7. Recalcula áreas cobertas e conflitos.
8. Chama a especialista adequada.

Regra rígida de saída

Uma resposta pode conter confirmação, contexto curto e opções, mas somente uma pergunta que pede resposta. Ela sempre termina com três opções prontas, `4. Avançar` e `5. Conversar mais sobre este tema`. Nunca apresente um formulário, duas perguntas encadeadas ou “responda A e B”. Se duas áreas estiverem pendentes, escolha a mais importante e deixe a outra para o próximo turno.

`Avançar` encerra a etapa com o entendimento atual. `Conversar mais sobre este tema` abre texto livre sobre a mesma pergunta e não autoriza uma segunda pergunta no turno.

Limite sobre fontes externas ao MVPFy

Specs, backlogs e documentos do projeto podem ser lidos como contexto. O MVPFy não cria, altera, renomeia, remove ou migra esses arquivos. O único documento final que ele escreve é `MVP.md`, além do estado em `.mvpfy/`.

Skill mvpfy-problem

Esta especialista transforma uma descrição de solução em um problema de negócio observável. "Preciso de um aplicativo" é uma entrada; a saída precisa explicar qual situação gera perda, atraso, erro, custo ou insegurança.

Analisa

- situação e frequência;
- pessoa afetada;
- tarefa que ela tenta realizar;
- impacto atual;
- alternativa usada hoje;
- motivo da insuficiência;
- comprovações já disponíveis;
- hipótese que o MVP deve testar.

Saída

```
Para [público], que precisa [tarefa], o problema é [dificuldade], causando [impacto]. Hoje usa [alternativa], que falha porque [limitação]. O MVP deve testar [hipótese].
```

Exemplo

"Agências perdem leads" ainda precisa de frequência, consequência e alternativa. Se o contexto existente já disser que os contatos chegam por três canais e são controlados em planilhas, a especialista usa essa informação e pergunta apenas o ponto decisivo que faltar.

Não faz

Não escolhe framework, não cria módulos e não define preço. Faz handoff para mvpfy-audience quando o grupo afetado estiver claro.

Skill `mvpfy-audience`

Esta especialista separa mercado amplo, segmento prioritário, usuário, comprador, pagador, influenciador e beneficiário. Ela evita personas genéricas como “qualquer empresa”.

Analisa

- frequência e intensidade do problema por segmento;
- acesso ao público;
- capacidade de pagar;
- comportamento atual;
- objeções e gatilhos;
- canais de comunicação;
- diferenças entre uso e compra.

Saída

O `MVP.md` recebe um segmento prioritário e, no máximo, três personas úteis: produto, compra e marketing quando forem diferentes. Cada persona tem contexto, objetivo, dor, alternativa, objeção, gatilho e canal.

Exemplo

Na ferramenta para agências, a agência pode ser compradora e pagadora, a pessoa de atendimento pode ser usuária diária e o cliente da agência pode ser um usuário com permissão limitada. Essa distinção afeta produto, SaaS, preço e marketing.

Não faz

Não inventa biografias para preencher espaço e não cria três públicos prioritários sem uma escolha. Faz handoff para `mvpfy-product` quando houver um segmento e um resultado central.

Skill mvpfy-product

Esta especialista converte problema e público em uma primeira versão utilizável e vendável. Sua unidade de trabalho é a jornada principal, não a lista de recursos desejados.

Analisa

- promessa central;
- entrada, processamento e saída;
- jornada de ponta a ponta;
- evento de ativação;
- módulos essenciais;
- perfis e permissões;
- regras de negócio;
- integrações indispensáveis;
- itens posteriores.

Classificação

CLASSE	USO
Essencial	Sem o item, a promessa não é entregue.
Necessária	Sustenta operação, segurança ou cobrança inicial.
Posterior	Pode esperar aprendizado.
Descartada	Não atende ao problema prioritário.

Exemplo

No caso dos leads, registrar um contato, atribuí-lo e mostrar seu andamento podem compor a jornada. Aplicativo móvel e marketplace não entram apenas porque seriam interessantes.

Não faz

Não define sozinho quem paga ou como a conta é criada. Encaminha essas dependências para `mvpfy-saas` e depois recebe as restrições de volta.

Skill mvpfy - saas

Esta especialista define como o software se comporta como serviço recorrente. Ela completa a jornada funcional com a jornada comercial e operacional.

Analisa

- B2B, B2C ou profissionais independentes;
- titular da conta e usuários;
- separação dos dados entre contas;
- convite, demonstração ou teste;
- onboarding e primeiro valor;
- ativação;
- unidade de valor;
- assinatura, limites e excedentes;
- cancelamento, inadimplência e suporte;
- processos manuais aceitáveis durante a validação.

Exemplo

Uma agência pode criar a conta, adicionar seus clientes e manter a cobrança em seu próprio contrato. Os clientes acompanham os leads, mas não enxergam dados de outras contas.

Recomendação padrão

Para muitos primeiros produtos, uma aplicação compartilhada com separação lógica segura é suficiente. Instância dedicada exige justificativa de segurança, contrato, desempenho ou posicionamento.

Não faz

Não presume que todo SaaS precisa de billing totalmente automático no primeiro dia. Se a operação assistida validar a oferta, ela pode ser registrada como provisória.

Skill mvpfy-brand

Esta especialista prepara a direção da marca depois que problema, público e promessa têm base suficiente. O resultado orienta naming e comunicação, sem afirmar disponibilidade de domínio ou marca registrada sem pesquisa.

Entrega

- categoria;
- posicionamento em uma frase;
- benefício central;
- diferenciais;
- personalidade e tom;
- palavras a usar e evitar;
- critérios para o nome;
- sugestões de nome quando solicitadas;
- slogans curtos;
- direção visual conceitual.

Exemplo

Se o valor for dar visibilidade aos leads para agências pequenas, o posicionamento deve falar de acompanhamento e resultado, sem depender de uma promessa genérica de inteligência artificial.

Não faz

Não cria logo final, não verifica disponibilidade automaticamente e não deixa o nome substituir a definição do problema.

Skill mvpfy-market

Esta especialista pesquisa concorrentes e alternativas quando você fornece referências ou pede dados atuais. Ela registra fatos observados com fonte e data e separa inferência de informação publicada.

Analisa

- concorrente direto, indireto e alternativa manual;
- público e promessa;
- recursos comparáveis;
- preço, moeda, periodicidade e limites;
- lacuna percebida;
- condições promocionais ou preço sob consulta.

Saída

Uma tabela no `MVP.md` com referência, tipo, público, promessa, recursos, preço, lacuna, link e data da consulta.

Exemplo

Se um concorrente mostra “US\$ 49 por mês para cinco usuários”, isso entra como preço observado com sua moeda e limite. Se a página só diz “fale com vendas”, o documento registra preço não publicado.

Não faz

Não inventa preço, não trata uma página antiga como atual e não inclui um recurso no MVP apenas porque um concorrente o possui.

Skill mvpfy-pricing

Esta especialista transforma valor, custo e comportamento de compra em uma hipótese comercial testável. Ela prefere o modelo de cobrança que acompanha o valor percebido sem criar medição desnecessária.

Analisa

- comprador e pagador;
- unidade de valor;
- ticket imaginado;
- preço de alternativas;
- benefício financeiro ou operacional;
- custo fixo e variável;
- suporte e esforço de venda;
- implantação, teste e inadimplência;
- MRR, ARPA e clientes para equilíbrio.

Saída

O plano apresenta faixa de preço, premissas, modelo de cobrança e cenários enxuto, base e crescimento. Quando os dados são insuficientes, a especialista marca a faixa como hipótese.

Exemplo

Se o valor nasce da gestão de vários clientes, cobrar por conta da agência pode ser mais simples do que cobrar por cada lead no começo. Essa é uma recomendação a validar, não uma regra universal.

Não faz

Não oferece precisão contábil, não promete margem e não fixa o preço sem considerar custo operacional e disposição de pagamento.

Skill mvpfy - technology

Esta especialista traduz a jornada do MVP em uma arquitetura pequena, operável e coerente com a fase da empresa.

Base recomendada

- Laravel como aplicação principal;
- Node.js somente com justificativa concreta;
- PostgreSQL, com Supabase quando fizer sentido;
- VPS;
- armazenamento compatível com S3;
- e-mail transacional;
- IA externa apenas com função de produto clara;
- backups, logs e tarefas agendadas conforme necessidade.

Entrega

O `MVP.md` recebe arquitetura, dados conceituais, autenticação, papéis, contas, isolamento, integrações, filas, arquivos, segurança básica, observabilidade e custo mensal por cenário.

Exemplo

Para a ferramenta de leads, a arquitetura precisa separar agências e clientes, registrar mudanças de status e enviar notificações. Não precisa começar com microsserviços ou Kubernetes sem um motivo concreto.

Não faz

Não implementa o software, não compra VPS e não trata a stack padrão como requisito que supera o problema do produto.

Skill mvpfy-marketing

Esta especialista prepara a chegada do primeiro SaaS ao mercado. Ela conecta promessa, público, oferta, conteúdo, captação e venda em poucos canais que a empresa consiga manter.

Entrega

- mensagem principal;
- objeções e respostas;
- oferta inicial;
- website mínimo;
- canal prioritário;
- conteúdo por etapa;
- lead magnet quando fizer sentido;
- e-mail marketing;
- grupos e redes sociais;
- demonstração ou venda assistida;
- métricas de aquisição, ativação, conversão e retenção.

Exemplo

Para agências, o primeiro conteúdo pode mostrar como acompanhar um lead desde o contato até o retorno ao cliente. O canal só entra no plano se o público realmente estiver presente e a equipe conseguir manter a rotina.

Não faz

Não recomenda todos os canais, não cria campanha final e não define mensagem sem usar o público e a proposta registrados pelas especialistas anteriores.

Skill `mvpfy-document`

Esta skill renderiza e valida o arquivo final. Ela não decide o conteúdo dos domínios; recebe fatos, escolhas, recomendações, hipóteses e pendências da orquestradora e os organiza conforme o template.

Arquivos usados

- `assets/MVP.template.md` ;
- `scripts/render-company.mjs` ;
- `scripts/validate-company.mjs` ;
- `references/document-rules.md` .

Operação

```
node skills/mvpfy-document/scripts/render-company.mjs --project .  
node skills/mvpfy-document/scripts/validate-company.mjs --project .
```

O renderer localiza IDs estáveis, mantém conteúdo existente e usa "Pendente" quando ainda não houver informação. O validator verifica se as áreas mínimas do primeiro SaaS estão presentes.

Não faz

Não cria `spec.md` , não edita backlog e não apaga uma resposta para substituir por texto genérico.

Skill `mvpfy-migrate`

Esta skill atualiza um `MVP.md` quando o template evolui. Ela usa IDs de seção e versão de schema para inserir o que falta no lugar correto.

Fluxo

1. Ler a versão do documento.
2. Ler IDs existentes.
3. Comparar com o template atual.
4. Inserir seções ausentes.
5. Preservar o conteúdo preenchido.
6. Marcar campos novos como pendentes.
7. Atualizar a versão após gravação válida.
8. Devolver à orquestradora uma única pergunta nova relevante.

Exemplo

Se o template ganhar “Plano de onboarding” e o documento já descreve convite e primeiro valor em outra seção, o texto antigo permanece. A nova seção recebe uma síntese segura ou “Pendente”; o MVPFy pergunta somente o detalhe que não puder ser inferido.

Não faz

Não remove histórico, não duplica seções, não migra arquivos do Specsify e não faz uma rodada de perguntas em lote.